

SKILL-01

To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile

1. Installation of Linux VM

- A Linux Virtual Machine is installed using virtualization software such as VirtualBox.
- Ubuntu Linux is installed and configured inside the VM.
- The terminal is used to perform Linux commands and development tasks.
- The Linux installation is verified using the `uname` command.

2. GCC Configuration

- GCC (GNU Compiler Collection) is used to compile C programs in Linux.
- GCC is installed using the Ubuntu package manager if it is not already available.
- The GCC installation is verified using the `gcc --version` command.
- C source files are compiled using GCC and executed in the Linux environment.

3. Git Repository Setup

- Git is used for version control and to maintain the project source code.
- Git is configured with the user's name and email address.
- The GitHub repository is cloned into the Linux environment.
- Project files and changes can be tracked using Git.
- The repository status can be checked using the `git status` command.

4. Project Structure Creation

- A main project directory named **ForgeOS** is created.
- Separate directories are created for different components of the project.
- The shell directory is used to store shell-related source files.
- The kernel directory is used to store kernel-related source files.
- A proper directory structure makes the project organized and easier to maintain.

5. Understanding Shell Architecture

- A shell acts as an interface between the user and the operating system.
- It accepts commands entered by the user and interprets them.

- The shell uses system calls to request services from the operating system kernel.
- `fork()` is used to create a new process.
- `exec()` is used to execute a command.
- `wait()` allows the parent process to wait for the child process.
- The basic architecture is:
User → Shell → System Calls → Kernel → Hardware

6. Building the Initial Makefile

- A **Makefile** is used to automate the compilation of the project.
- It contains rules that specify how source files should be compiled.
- The Makefile reduces the need to type long compilation commands repeatedly.
- The `make` command reads the Makefile and performs the required compilation.
- An initial Makefile is created for compiling the ForgeOS shell component.

To Analyze process abstraction, execute `fork()`, understand `exec()` family, analyze parent-child relationships, inspect process tree, practice system call tracing.

1. Analyze Process Abstraction

- A process is a program that is currently being executed.
- The operating system represents each process using a process control structure.
- Each process has information such as PID, PPID, state, and other execution details.
- The OS provides an abstraction that allows multiple processes to execute independently.
- Process information can be observed using commands such as `ps` and `top`.

2. Execute `fork()`

- `fork()` is a system call used to create a new process.
- The process that calls `fork()` is called the **parent process**.
- The newly created process is called the **child process**.
- The child process receives a new PID.
- `getpid()` can be used to obtain the process ID.

- `getppid()` can be used to obtain the parent process ID.
- The parent and child processes can execute independently after `fork()`.

3. Understand the `exec()` Family

- The `exec()` family of system calls is used to execute a new program.
- It replaces the current process program with another program.
- Common functions include `execl()`, `execlp()`, `execv()`, and `execvp()`.
- After a successful `exec()` call, the new program runs using the same process ID.
- `fork()` and `exec()` are commonly used together to create a child process and execute another program in it.

4. Analyze Parent-Child Relationships

- A parent process can create one or more child processes using `fork()`.
- Each child process has its own PID.
- The child's PPID identifies its parent process.
- `getpid()` displays the current process ID.
- `getppid()` displays the parent process ID.
- Parent-child relationships can be examined using process information commands.

5. Inspect the Process Tree

- A process tree shows the hierarchical relationship between processes.
- It helps identify which process created another process.
- The process tree can be viewed using commands such as:

`pstree`

- The `ps` command can also be used to examine process relationships.
- Process trees are useful for understanding parent and child process creation.

6. Practice System Call Tracing

- `strace` is a Linux utility used to trace system calls made by a program.
- It helps observe how a program interacts with the Linux kernel.
- A program can be traced using:

`strace ./a.out`

- System calls such as `fork()`, `execve()`, `read()`, `write()`, and `wait()` can be observed depending on the program.
- System call tracing helps understand the interaction between user programs and kernel services.